

Here's how to spin up Elasticsearch locally in Docker so you can run through the exercise end-to-end.

Option 1 — docker-compose (better if it needs to sit alongside Kafka Connect)

Since you're already running Kafka/Connect, it's cleaner to put everything on one Docker network so container names resolve to each other.

#docker-compose.yml

```
version: "3.8"

services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.15.0
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
    ports:
      - "9200:9200"
      - "9300:9300"
    networks:
      - spark-net

  kibana:
    image: docker.elastic.co/kibana/kibana:8.15.0
    container_name: kibana
```

```
environment:
  - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
ports:
  - "5601:5601"
depends_on:
  - elasticsearch
networks:
  - spark-net
```

```
networks:
  spark-net:
    external: true # reuse the network your Kafka/Connect containers are on
```

Start it:

```
docker compose up -d
```

Check the network name your Kafka stack is already using:

```
docker network ls
docker inspect kafka0 | grep -A5 Networks
```

If your Kafka Connect container is on a compose-created network like `kafka_default`, either put this `elasticsearch` service in the **same compose file** as Kafka/Connect (simplest), or set `networks.kafka-net.name: kafka_default` to attach to the existing one.

1 Adjusting your connector config for Docker networking

If Kafka Connect is running in its own container, `http://localhost:9200` from inside that container refers to *itself*, not your host — so the sink connector config needs to point at the Elasticsearch **container name** instead:

```
"connection.url": "http://elasticsearch:9200"
```

(This only matters if Connect is containerized. If Kafka Connect is running directly on your host machine and only Elasticsearch is in Docker, `localhost:9200` is fine as-is since the port is published to the host.)

Sanity checks before creating the connector

```
# ES is healthy
curl http://localhost:9200/_cluster/health?pretty
curl http://elasticsearch:9200/_cluster/health?pretty
```

```
# From inside the Connect container, ES is reachable by container name
docker exec -it <connect_container_name> curl http://elasticsearch:9200
```

Once both of those succeed, go ahead and re-run the `POST /connectors` call from the earlier exercise — just update `connection.url` if Connect is containerized.